

Evolutionary Algorithms for a Better Gaming Experience in Rehabilitation Robotics

KLEBER O. ANDRADE, RICARDO C. JOAQUIM, and GLAUCO A. P. CAURIN, University of São Paulo
MARCIO K. CROCOMO, Federal Institute of Education, Science and Technology of São Paulo

This article proposes the use of two evolutionary algorithms (EAs) to the dynamic difficulty adjustment (DDA) of a serious game in the rehabilitation robotics application. DDA occurs in runtime for a better user experience with a game. This approach is used to improve the quality of the game experience and to avoid boredom or frustration for players with severe limitations imposed by pathologies such as stroke, cerebral palsy, and spinal cord injuries. The first EA solves the game adjustment problem, changing the game difficulty according to the player's skill, and the purpose of the second EA is to adjust the coefficients of the first EA's objective function so that it can work in a more effective way. To do so, the second EA uses results of game matches against simulated player profiles. The results shows that the presented method was able to identify a set of coefficients that allows the first EA to correctly adjust the difficulty level for all six tested player profiles.

CCS Concepts: • **Theory of computation → Evolutionary algorithms**; • **Computing methodologies → Artificial intelligence**; • **Software and its engineering** → *Interactive games*;

Additional Key Words and Phrases: Evolutionary algorithms, serious games, rehabilitation robotics, artificial intelligence

ACM Reference format:
Kleber O. Andrade, Ricardo C. Joaquim, Glauco A. P. Caurin, and Marcio K. Crocomo. 2018. Evolutionary Algorithms for a Better Gaming Experience in Rehabilitation Robotics. *ACM Comput. Entertain.* 16, 2, Article 4 (April 2018), 15 pages.
https://doi.org/10.1145/3180657

1 INTRODUCTION

Games with dynamic difficulty adjustment (DDA) are the subject of many studies [4, 5, 17, 18, 28, 29, 33, 35]. It is common knowledge that some games become boring when they are too easy, or frustrating when too hard to play. The first classical setting for a game difficulty is to allow the player to select a challenge level, usually easy, medium, and hard; however, those are static configurations and do not allow the player to evolve during his playing journey [20].

The user's performance is affected by the difficulty of the game and therefore may induce a variety of cognitive and emotional states [16]. Dealing with players of different skill levels, and

This work was supported by FAPESP, CNPq, and CAPES. We would like to especially thank Anderson Matheus, Jeff Simmons, and Alan Prado for helping to develop "'The Catcher"' game.
Authors' addresses: K. O. Andrade (corresponding author), R. C. Joaquim, and G. A. P. Caurin, Mechanical Engineering Department, University of São Paulo; emails: {pdjkleber, rcjoaquim}@gmail.com, gcaurin@sc.usp.br; M. K. Crocomo, Computer Science Department, Federal Institute of Education, Science and Technology of São Paulo; email: marciokc@gmail.com.
Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than ACM must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
© 2018 ACM 1544-3574/2018/04-ART4 $15.00
https://doi.org/10.1145/3180657

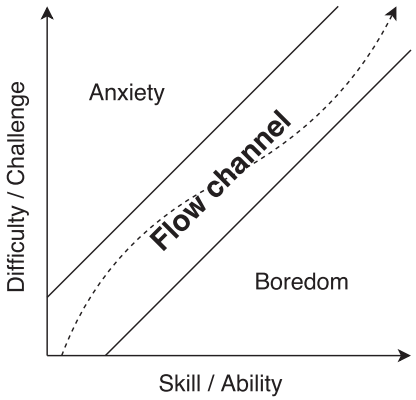


Fig. 1. Flow channel diagram.

their different abilities to learn and adapt during a game, is one of the characteristics that game developers care about when designing a game—one that can be the difference between a good and a bad game [3]. If the game can automatically balance the offered challenge level according to the player's skill level, it can stimulate the player to keep his or her attention on the proposed activity [30] while maintaining the user in a so-called flow state [27].

In Braga and Marinho [8], the *flow* [11] applied to games is described as a possibility to measure the fun. Cowley et al. [10] remind participants that they experiment with flow when there is a balance between challenges and abilities. Figure 1 show two dimensions: degree of challenge and skill. Within those, there are three regions: anxiety, boredom, and flow. If the challenge is at a high level and skills at a low level, the individual gets anxious. If the challenge is at a low level and the skills at a high level, the individual gets bored, Between the two dimensions, if there is challenge/skill compatibility, the subject experiences the flow.

Several techniques are used to solve the DDA problem, such as hybrid position/force fuzzy logic controllers [21], artificial neural network [9], reinforcement learning with Q-learning [4], adaptive systems [34], the GENTLE/s system [2], adaptive algorithms employing the speed-accuracy trade-off [26], and controller POMDP (partially observable Markov decision process) [23]. As some other authors have done [3, 7, 36], this article proposes adjusting the difficulty level of a game, but this time optimizing the difficulty balance strategy using two evolutionary algorithms (EAs). More precisely, this article proposes an optimization of the difficulty balance strategy during the game using EAs. The first EA [5] solves the DDA problem in games for robotic rehabilitation. The second EA adjusts the coefficients utilized by the fitness function of the first EA for a better user experience. The optimization results were obtained in this context based on matches played by simulated player profiles.

2 MATERIALS AND METHODS

This section presents the materials and methods used to perform the dynamic adjustment of difficulty in games applied to rehabilitation processes aided by robots. In this context, we developed two EAs:

- The first EA (EA1) uses the weighted-sum method to adjust the difficulty of the game in real time (Section 2.4).
- The second EA (EA2) optimizes the coefficients of the fitness function of EA1 (Section 2.5).

To avoid the need to subject humans to a large number of experiments for fitting and validating the EAs, we created a simplified one-player model using a fifth-order polynomial (Section 2.3).

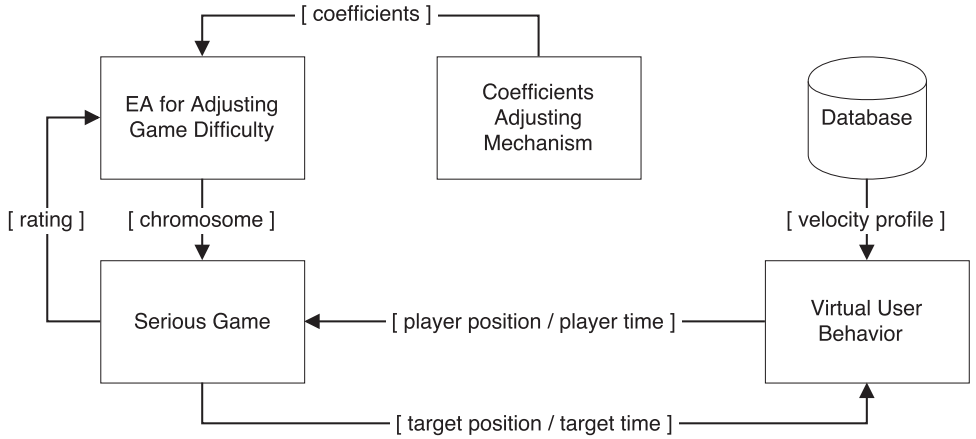


Fig. 2. Summary of the proposed approach.

This model also allows for a greater number of interaction tests with the game, reducing the time and costs involved with the research activities. The game called "The Catcher" has some game space with the definition of the relevant variables presented at Section 2.2. Robot hardware used is present in Section 2.1, following the synthesis of the method used that is displayed in Figure 2.

The architecture used for game-based rehabilitation robotic follows the guideline of Kamkarhaghighi et al. [22].

2.1 Robot Description

The robot (Figure 3) was designed to perform active and passive therapy [6, 13]. Therapy is classified as active when only the patient is responsible for the efforts that result in movements—for instance, the work performed by the robot is null ($WR = 0$). Passive therapy occurs when both the robot and patient produce efforts that result in movements; in this case, the robot provides work ($WR \neq 0$). For passive therapy, our system uses impedance control, modifying the motor control system stiffness to react according to the patient force reaction. In this article, we only use the robot for active therapy.

The device is designed for therapy exercises with one of following single degrees of freedom (DoF) at a time: flexion/extension, adduction/abduction, or pronation/supination. The choice for only one DoF is supported by evidence that simultaneous therapy exercises with more than one do not provide significant improvements when compared to single movement exercises [25]. Some mechanical settings are responsible for this setup. The structure has two aluminum links connected to a rotatory joint, where the first is connected to a bracket and the second one carries the input handle.[1] The first bracket is attached to the user's forearm during the therapy sessions. The device uses a DC motor from Maxon Motor Company (RE40 #148867) that is directly coupled to the rotor axis, using no gear boxes, which provides the system with backdriveability.

With regard to the electrical hardware and sensors, the servomotor has an integrated 2,000 pulses per turn (ppt) encoder and is connected in low level to the control system, implemented using a Maxon EPOS 24/5 control driver. A CANOpen[2] network links the information exchange between the servomotor control driver and the communication layer.

[1]Commercial Bracing Axilo Palmar Salvape R Lite Supp.
[2]http://www.can-cia.org/.



Fig. 3. Wearable robot to play game.

The communication layer, between the game and the robot, is implemented using TCP/IP sockets (Transmission Control Protocol). This server runs on a Toradex Colibri T20 computer with an ARM Cortex A9 processor and a Windows Embedded Compact 7 RTOS. It acts as a middleware[3] monitoring the changes in the robot joints and reporting them to the client (game) at an update rate of 200Hz. The client (MAC Mini 2.3GHz quad core with 4GB, 1.6MHz SDRAM DDR3, and an Intel HD Graphics 4000 graphics card) renders model images at 60Hz.

Data transmission is implemented serializing a structure of messages from robot-to-game and game-to-robot. The robot-to-game message structure is composed of only two variables: status (integer variable that informs about the connection) and position (double variable that gives the robot axis movement). The game-to-robot message structure is composed of the following variables: control (integer variable), set point (double variable), stiffness (double variable), dumping (double variable), left window (double variable), and right window (double variable). Conceptually, the device is simple and portable, so it is some prototype aiming toward the development of home therapy devices.

2.2 Game Description

The Catcher game[4] was developed for robotic rehabilitation (Section 2.1) in combination with single DoF robots [5, 13]. The game is then used to perform the difficulty balancing experiments using EA1 (Section 2.4) and EA2 (Section 2.5). The objective of the player in The Catcher is to control the squirrel (game character) toward falling hazelnuts (target object). The player should direct the character to collect as many objects as possible during a therapy session. The character

[3]Middleware usually describes a software binder, or mediator, between two existing and independent program codes. Its function provides the applications with the independence of transmission systems.
[4]The Facebook game page is available at https://www.facebook.com/thecatcher4rehab.



(a) (b)

Fig. 4. Screenshot of the developed game, The Catcher. (a) The player was able to take the character to the target object and is waiting for him to reach the basket to score a point. (b) The player failed to get the character to come to the hazelnut, then the target object hits the ground, and it breaks down, causing dust to rise.

only moves in the x-axis of the screen, corresponding to the flexion of the wrist (palmarflexion) or wrist extension (dorsiflexion). Figure 4 presents two screenshots of the game.

When starting the game, the health professional (therapist or physiotherapist) sets a fixed amount of N_t target objects to be collected by the player. A single target object, coordinate (x_t, y_t), appears on the screen at a time, and it always appears at a distance d_t from the player whose position is defined by (x_p, y_p). The x_p and y_p coordinates respectively refer to the player's center point and x_t and y_t to the target's central point, being all coordinate values measured in pixels. The object has the constant vertical velocity of fall v_t (gravity acceleration is not considered). When a target object is captured or lost, a new target is generated. If the player can reach the target object before it hits the ground, he scores a point. The time interval between two targets is called a *round* of the game.

Another game variable defined by r_p is the distance from the center (x_p, y_p) in pixels to any point in the circumference that delimits the character on the screen. The squirrel is not the object of interest, so the radius is related to the basket because it collects the target object that falls. r_t is the distance from the target's center (x_t, y_t) in pixels to any point in the circumference that delimits the target object on the screen. The display size it is given by $w \times h$, where w is the width of the world expressed in pixels. w equals the maximum screen resolution width, and w_l and w_u respectively refer to the lower and upper limits of the display width. The h variable represents the height of the world given in pixels. h equals the maximum screen resolution height to the squirrel's foot, and h_l and h_u respectively refer to the lower and upper limits of the display height.

To define where the target object will appear on the screen, a random number, $rnd \in [0, 1]$, is generated and determines the probability of the target's direction in relation to x_t. If rnd is less than or equal to 0.5, the target object will be thrown to the left ($dir_t = -1$), and otherwise it will be thrown to the right ($dir_t = 1$) from the player, since $x_p + d_t * dir_t \in (w_l, w_u)$. Next, the final position of the object target $(x_t, y_t = h_u)$ and its displacement time is given by $t_t = \frac{h_u - h_l}{v_t}$. Finally, update the player's position according to the model presented in Section 2.3, and check for collision $|x_p - x_t| \leq r_p + r_t$ between the player area and the area of the target object, increasing the player's score. The whole process then is repeated until all N_t target objects are tested, obtaining, in the end, a score and the playing time used in the therapy session.

2.3 Virtual User Behavior

This section presents a motion behavior model for a user's single DoF. In our case, the rehabilitation is performed just for flexion and extension movement of the user's wrist joint. This movement

representation is based on the definition of discrete movements performed by healthy individuals [19]. This model generates velocity profiles that approximate human movements between two points.

For simplicity reasons, currently, our work is not concerned with the generation of reliable copies of human movements. In this article, the benefits provided by the adoption of a virtual player are presented. Future work will address the challenge of reproducing human behavior with higher accuracy. In this context, the dynamic movement primitives (DMP) approach will probably replace the approach based on polynomials [32].

This article focuses on the establishment of a software tool that allows the integration of different models of movement. Modularity is one of the important features of this approach. It allows, shortly, the use of submovements resulting in emulation of complex behaviors of some forms of motor disability [31]. The generation and study of more complex movements will be the subject of future work.

A fifth-order polynomial $x(t)$ was adopted to simulate two motion primitives—that is, the user-introduced motion in the game and an ideal move that would be required to perform a task presented in a round about the game. These polynomials represent how the robot handles the trajectory as a function of time from the player's initial position $x_i = 0$ to the target position x_d.

$$x(t) = c_0 + c_1 t + c_2 t^2 + c_3 t^3 + c_4 t^4 + c_5 t^5 \tag{1}$$

Polynomial coefficient values c_i are defined by the introduction of user movement characteristics (e.g., the trajectory duration time t_f). The velocity of user joint motion at starting time t_0 and at the velocity end of the movement are assumed to be zero ($v(0) = 0$ and $v(t_f) = 0$).

$$v(t) = c_1 + 2c_2 t + 3c_3 t^2 + 4c_4 t^3 + 5c_5 t^4 \qquad 0 \le t < t_d \tag{2}$$

$$v(t) = 0 \qquad t \le 0 \tag{3}$$

$$v(t) = 0 \qquad t \ge t_d \tag{4}$$

It is also assumed that the user reaches the maximum characteristic velocity at the mean displacement time $t_m = \frac{t_f - t_i}{2}$. To provide a more realistic movement, a time delay was added to the trajectory function by considering a patient's reaction time τ and a deviation position $\xi = N(m, \sigma)$, assuming a distribution Normal $N()$, with mean of m and standard deviation σ. Then the following coefficient values are obtained analytically:

$$\begin{cases} c_0 = \dfrac{(x_d + \xi)\tau^3(10t_d^2 - 5t_d + \tau^2)}{(t_d - \tau)^5} \\ c_1 = -30\dfrac{t_d^2(x_d + \xi)\tau^2}{(t_d - \tau)^5} \\ c_2 = 30\dfrac{t_d(x_d + \xi)\tau(t_d + \tau)}{(t_d - \tau)^5} \\ c_3 = 10\dfrac{(x_d + \xi)(t_d^2 + 4t_d\tau + \tau^2)}{(t_d - \tau)^5} \\ c_4 = -15\dfrac{(x_d + \xi)(t_d + \tau)}{(t_d - \tau)^5} \\ c_5 = 6\dfrac{(x_d + \xi)}{(t_d - \tau)^5} \end{cases} \tag{5}$$

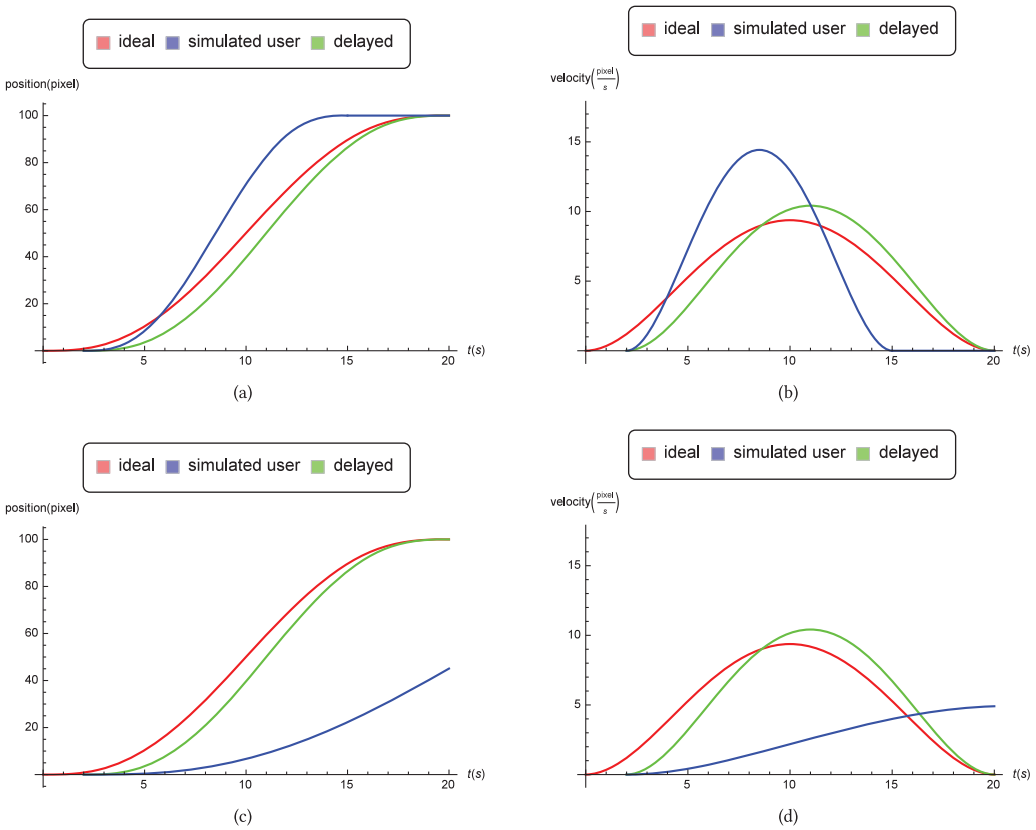


Fig. 5. Examples of trajectories using the parameters $t_i = 0$, $x_d = 100$, $t_f = 10$, and $\tau = 2$: (a) Position versus time curves with user maximal velocity $v_m = 13.8$ show the player reaching the target ahead of time. (b–d) For the same user maximal velocity, the diagram presents bell-shaped velocity curves over time (b); with $v_m = 4.7$, the player misses the target object (c) and the corresponding velocity curves' bell shape (d).

The player's position deviation from the target $\xi = N(0, 1)$ modeled as a normal distribution with zero mean and a standard deviation of 1. Similarly, user reaction time is modeled as a time delay $\tau = N(m_\tau, \sigma)$ that also assumes values defined by a normal distribution.

Comparing an ideal trajectory with the player's trajectory, it is possible to generate behaviors where the user reaches the target object in advance waiting for the target object and also actions when the player does not achieve the target object in the available time. Figure 5 presents two example situations: for each situation, there are two graphs—the graph of the player's movement over time and the player's velocity over time. The examples were chosen to illustrate the capacity of the system to reproduce conditions that are commonly observed in clinical trials with real users. For instance, sometimes the user arrives ahead of the ideal time, but in other moments, the player does not reach the target position on time.

2.4 Adjusting Game Difficulty

In a previous work of the authors [5], an EA based on an optimization strategy [24] was adopted to adjust game difficulty to the user model. A three-step approach was implemented. In the first step, a random population with $N_{C_{EA1}}$ candidate solutions (chromosomes) is created. The second step

evaluates the candidate solutions, and the third step consists of creating a new set of candidate solutions by making copies of $N_{S_{EA1}}$ selected chromosomes with the highest fitness values and applying the Creep mutation operator [12]. Game adaptation runs in a loop between steps 2 and 3, until the end of a game session. A detailed explanation of this EA is available in Andrade et al. [5].

Although the three-step approach is the same, the experiments described in the remainder of this article use a variation of the EA from Andrade et al. [5], which we will refer to as EA1.

The EA1 population initialized in the first step is organized in a matrix composed by $P_{i,j_{EA1}}$ elements. Each row assumes the array form $P = [d, v]$, where d and v are numbers between 0 and 1, given by Equations (6) and (7).

$$d = \frac{d_t - d_l}{d_u - d_l} \tag{6}$$

$$v = \frac{v_t - v_l}{v_u - v_l} \tag{7}$$

The terms d_l, d_u respectively are lower and upper bounds for the distance allowed by the game between the player and the target; v_l and v_u are lower and upper bounds for the target velocity. When we have the values of a chromosome (d and v), we can calculate d_t and v_t by isolating those terms in Equations (6) and (7). This way, the values of a chromosome are the necessary parameters to configure a game match and represent its difficulty. Therefore, $|P|$ corresponds to the number of variables that are used to calculate game difficulty, in this version, $|P| = 2$. Given an initial population of $N_{C_{EA1}}$ chromosomes, $N_{C_{EA1}} \times |P|$ elements are initialized with random values in the first step.

The second step, the evaluation of the candidate solutions in the population, uses the following fitness function (F_{EA1}), and the chromosomes with the highest fitness values are selected for the third step.

$$F_{EA1} = K_d \cdot d + K_v \cdot v - K_\epsilon \cdot \epsilon \tag{8}$$

$$\epsilon = \frac{|x_t - x_p|}{d_u} \tag{9}$$

In Equation (8), ϵ is the error module, calculated as the difference between the target position (x_t) and the player position (x_p), divided by d_u, as shown in Equation (9); K_d, K_v, and K_ϵ are coefficients that set the weight of the fitness elements. To function properly, this approach for adjusting game difficulty requires a proper adjustment of the coefficients in Equation (8). As an example, using a very high value for K_ϵ when compared to the other coefficients may result in the EA converging for solutions considered easy matches for the player, causing the player to get bored. However, using a small value for k_ϵ may result in solutions considered too hard, causing the player to get frustrated. For this reason, in the next section, we present a strategy for optimizing the values of K_d, K_v, and K_ϵ.

2.5 The Coefficients Optimization Strategy

The proposed coefficient adjustment mechanism consists of the development of a second EA (EA2) with the purpose of finding good values for the presented coefficients, adjusting the fitness of EA1 to achieve a better gaming experience. EA2 is organized in a similar way as the previous one. The chromosome is composed of an array of coefficients $K = [K_d, K_v, K_\epsilon]$ used in EA1 fitness function. Each coefficient is a real value in $[0, 1]$.

The first step of EA2 is to initialize a random population with $N_{C_{EA2}}$ chromosomes. After that, each chromosome is evaluated by actually using its parameters in EA1, responsible for adapting

the game difficulty in a sequence of matches using a set of simulated player profiles (Section 2.3). For each player profile, a sequence of $N_{samples}$ runs is performed, each one simulating $N_{matches}$ game matches. After that, we calculate the rate of matches in which the player has managed to reach the target, which we call the player's skill level (L_S) and the average game's difficulty (L_d), respectively calculated by Equations (10) and (11):

$$Ls = \frac{\sum_{k=1}^{N_{matches}} H_k}{N_{matches}}, \tag{10}$$

where Ls is the average score obtained and H_i is the score in each match k (1 if the target was reached before hitting the ground and 0 otherwise), and

$$Ld = \frac{\sum_{k=1}^{N_{matches}} \frac{d_k + v_k}{2}}{N_{matches}}, \tag{11}$$

where Ld is the average of the game difficulty evaluated for each sample using the coefficients of the chromosome being evaluated, and d_k and v_k are distances and speed of the target in each match k, which results in the game difficulty for a given match (Section 2.4).

Now that we know how to estimate the player skill level (L_s), and the degree of challenge (Ld) of the game for a given set of coefficients, we can see how close to the flow channel this set of coefficients will direct our game by plotting a point in a flow channel diagram, such as the one in Figure 1. If we consider a line passing in the center of the flow channel area, we can calculate the distance of the plotted point (Ls, Ld) to this line (represented by the equation of a line $ax + by + c = 0$) as being $\frac{|a \cdot Ls_{i,j} + b \cdot Ld_{i,j} + c|}{\sqrt{a^2 + b^2}}$. Since we want our game to be as close to the flow channel as possible while considering different player profiles, the objective problem of EA2 is to find a set of coefficients that minimizes its fitness function (F_{EA2}), which is calculated as the average distance of the points ($L_{s_{i,j}}, Ld_{i,j}$) to the flow channel line, where $L_{s_{i,j}}$ and $Ld_{i,j}$ respectively are the player skill level and game difficulty obtained in sample j of the tested player profile i, as shown in Equation (12). The goal of EA2 is to find chromosomes that minimize the fitness function presented in Equation (12) and, by doing so, find solutions that are closer to the flow channel, avoiding boredom or frustration [27].

$$F_{EA2} = \frac{\sum_{i=1}^{N_{profiles}} \sum_{j=1}^{N_{samples}} \frac{|a \cdot Ls_{i,j} + b \cdot Ld_{i,j} + c|}{\sqrt{a^2 + b^2}}}{N_{profiles} \cdot N_{samples}} \tag{12}$$

To create a new population $C2'$, each new chromosome is obtained by selecting two parents from the current population $C2$, using a size T deterministic tournament selection [1, 14], and combining them using the Blend crossover operator (BLX-α) [15].

BLX-α is a well-known crossover operator used for two parent solutions ($CS2_1$ and $CS2_2$); the BLX-α method randomly picks a solution in the range $[0, 1]$. An offspring is created as follows:

$$P_{offspring} \begin{cases} CS2_2 - \alpha(CS2_1 - CS2_2) & \text{if } CS2_1 \geq CS2_2, \\ CS2_1 - \alpha(CS2_2 - CS2_1) & \text{otherwise.} \end{cases} \tag{13}$$

After obtaining the new chromosome, each coefficient has a mutation chance (M) to be replaced by a new random value (a valid value in [0,1]). We also keep a copy of the chromosome with the best fitness from $C2$ to $C2'$. After a new generation is obtained as explained, it is evaluated again and the process is repeated for G_{EA2} generations.

Table 1. Commons Parameters for All Experiments

Description	Variable	Value
Population size of EA1	$N_{C_{EA1}}$	5
Number of repetitions of EA1	$N_{samples}$	30
Number of generations of EA1	G_{EA1}	200
Mutation range of EA1		0.1
Seed random		15
Width screen	w	1,920
Height screen	h	1,080
Population size of EA2	$N_{C_{EA2}}$	20
Number of generations of EA2	G_{EA2}	50
Mutation rate of EA2	M	0.05
α from EA2 BLX-α	α	0.5
Tournament size of EA2	T	2
Elitism of EA2		0.2
Constant a of F_{EA2}	a	1.0
Constant b of F_{EA2}	b	−1.0
Constant c of F_{EA2}	c	0.0

3 EXPERIMENTS AND RESULTS

We elaborated the experiments with two objectives. The first one was to verify the correct implementation of EA2, checking its capacity to adjust the coefficients of the first EA so that it can correctly adapt to a single player profile. For this purpose, the first three experiments are reported in Section 3.1 and use EA2 to find coefficients for three different profiles: players with (i) low, (ii) average, and (iii) high ability levels. The second objective of the experiments is to verify the capacity of EA2 to find coefficients that are able to adjust the difficulty for multiple player profiles, and with this purpose, experiments 4 and 5 are reported in Section 3.2.

All experiments reported in this section have the following parameters in common according to Table 1.

3.1 Experiments With Single Player Profiles

In experiments 1, 2, and 3, we run EA2 to find good coefficients to adapt the game difficulty for single player profiles, as enumerated next:

(1) *Experiment 1*: Player with low skill level
(2) *Experiment 2*: Player with average skill level
(3) *Experiment 3*: Player with high skill level.

Figures 6, 7, and 8 show the results. The left graph of each figure presents the fitness in each generation. In every case, the fitness values have a tendency to decrease as the generation number increases, as expected of a proper working EA with the goal of finding solutions with low fitness values. The right graph of each experiment is a scatter plot, in which each point represents the average difficulty of the game $(v_t + d_t)/2$ and the average skill level presented by the player (the fraction of total targets reached). The darkest points represent the results obtained using the coefficients of the population in the last generation of EA2, whereas the lighter points represent the previous generations. This way, we can perceive, in all scenarios, the convergence of the solutions to places that are near the line representing the flow channel, used in Equation (12). This shows

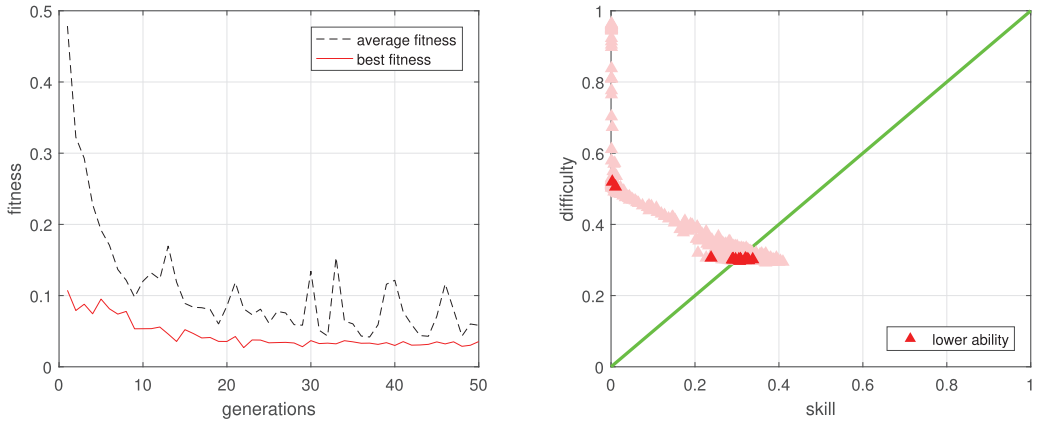


Fig. 6. Results of experiment 1. Fitness in each generation and the flow point for each gameplay. Coefficients found with the best individual of the last generation: $k_d = 0.423750$, $k_v = 0.074990$, and $k_e = 0.779430$.

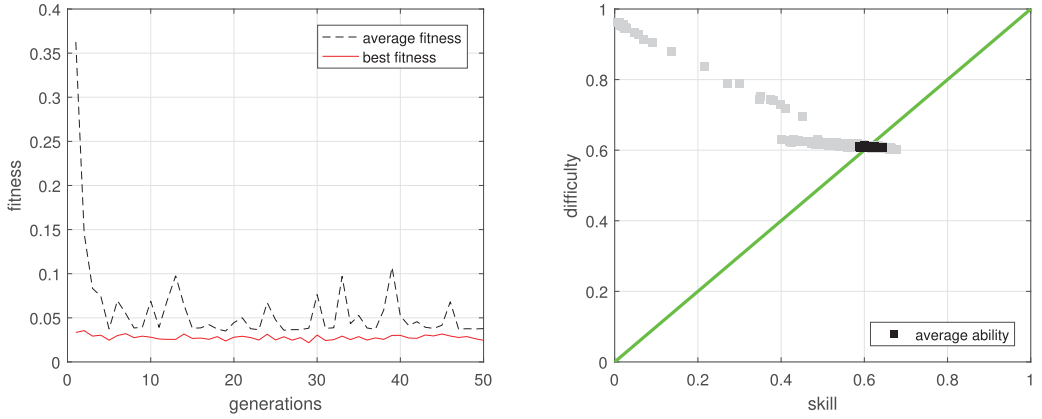


Fig. 7. Results of experiment 2. Fitness in each generation and the flow point for each gameplay. Coefficients found with the best individual of the last generation: $k_d = 0.101500$, $k_v = 0.494660$, and $k_e = 0.708550$.

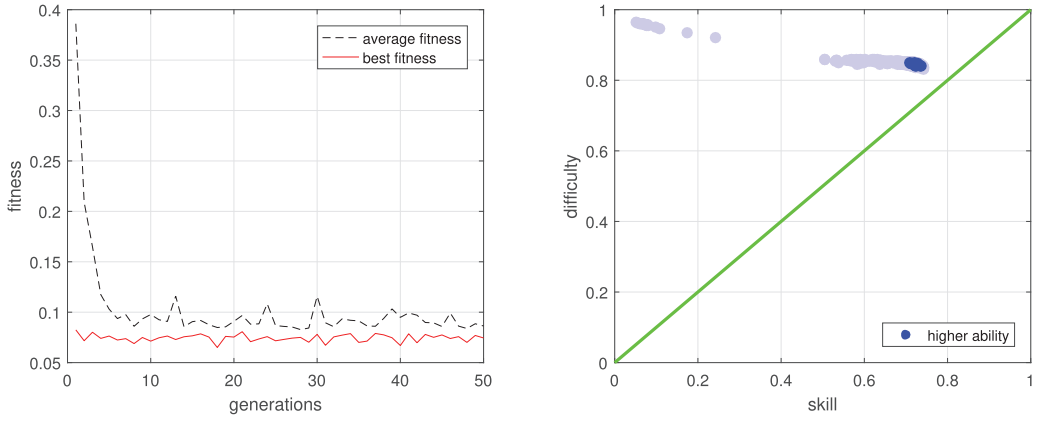


Fig. 8. Results of experiment 3. Fitness in each generation and the flow point for each gameplay. Coefficients found with the best individual of the last generation: $k_d = 0.017040$, $k_v = 0.415210$, and $k_e = 0.763860$.

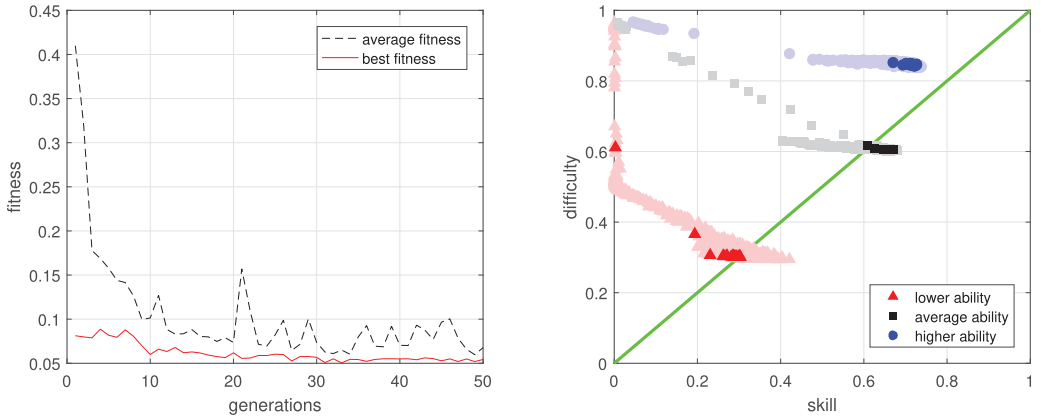


Fig. 9. Coefficients found with the best individual of the last generation: $k_d = 0.469780$, $k_v = 0.052540$, and $k_e = 0.781280$.

that EA2 is working properly, finding coefficients that allow the first EA to properly adapt game difficulty for a specific player profile.

3.2 Experiments With Multiple Constants Player Profiles

Each of the experiments presented so far found good coefficients for adapting the game for single player profiles (one set of coefficients was found for each profile). However, EA2 must be able to find coefficients so that the first EA is capable of adapting the game for different players. To verify if this is possible, the experiment was repeated using the three profiles (low, average, and high ability) at the same time to calculate the fitness for EA2, as shown in Equation (12). The results are presented in Figure 9, which shows that EA2 was able to find a set of coefficients capable of adjusting the game difficulty for three different player profiles.

3.3 Experiments With Multiple Variable Player Profiles

The previous experiment was repeated using six different profiles at the same time, with the purpose of finding one set of coefficients for adapting the game for a bigger number of players. Three new profiles were added to the constant profiles used so far: sine, square, and sawtooth profiles, which represent more complex player behaviors when compared to the constant profiles used previously.

The graphs in Figure 10 show that, again, EA2 was able to identify a set of coefficients that can properly adjust the game difficulty for all different player profiles used. We can also see that the dots representing the constant profiles in the last generation (darker points) are in similar regions when compared to the previous experiments (Figure 9). This means that the newly found coefficients are able to adapt the game for the new three player profiles without compromising the performance we had so far (when using the coefficients from the previous experiment) for the constant player profiles.

4 CONCLUSION AND FUTURE WORK

This article presents an EA that provides a better gaming experience in the context of rehabilitation robotics. As an optimization strategy, we adopted two EAs: EA1 and EA2. The parameters of the former are adjusted by the latter. This approach solves the DDA problem in a robotic rehabilitation

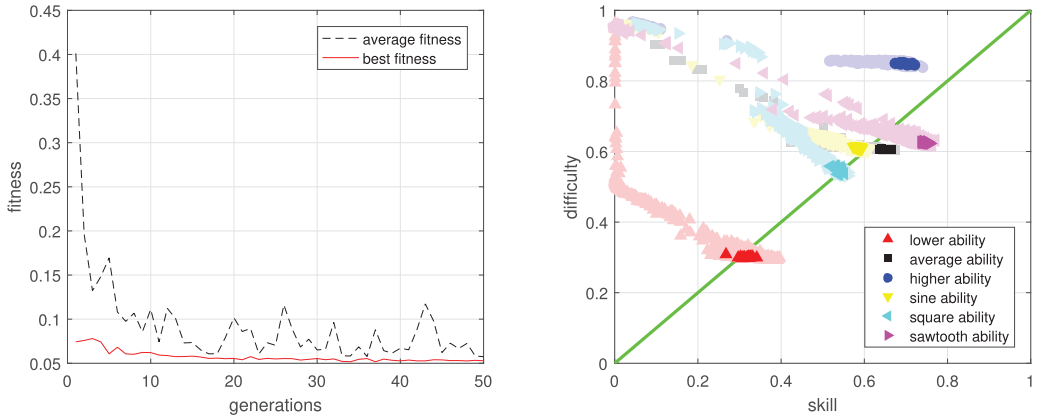


Fig. 10. Coefficients found with the best individual of the last generation: $k_d = 0.468190$, $k_v = 0.097790$, and $k_e = 0.883730$.

game. We also explain the central concepts, structure, and working principle of DDA. Its efficiency is demonstrated through simulated experiments.

The results obtained with the simulations of EA1 show that it is possible to adjust the game level according to the player's capacity. However, it is not certain that the difficulty level will be correctly adjusted for any set of coefficients used for the EA1 fitness function. The lighter-colored dots in the charts from Figures 6 through 10 that are far from the diagonal line (flow channel) show this uncertainty, as they represent scenarios in which EA1 could not adapt the difficulty level as desired. For EA1 to dynamically adjust the game level of difficulty for different player profiles in an efficient form, the fitness function coefficients must be properly adjusted. EA2 addresses this problem by improving these coefficients using simulated players in various experiments before the game is submitted to real players. Using optimized coefficients, EA1 can adapt game difficulty efficiently, as represented by the darker-colored dots in the charts from Figures 6 through 10. Consequently, the player is kept in a flow state, improving his therapy outcome.

The implementation and application of two integrated EAs represents one of the main contributions of this work to the research on artificial intelligence and serious games. Similar applications for EA may result in an interesting and original result. Contribution provided by the use of serious games in the context of robotic rehabilitation should also be emphasized, as most studies in the literature currently focus on the development and testing of robotic devices. Little attention has been given to the interaction of these systems with their users.

Our future work will focus on (i) the implementation and analysis of different genetic operators searching for alternative solutions to DDA; (ii) adding a power-based movement to the player's profile, taking the presence or absence of robot assistance into consideration; (iii) more exhaustive analyses of the parameter's influence on the EAs, looking for results optimization; and (iv) performing clinical tests with stroke subjects and evaluating the impact of our proposal on clinical rehabilitation sessions.

REFERENCES

[1] Bernardetta Addis, Andrea Cassioli, Marco Locatelli, and Fabio Schoen. 2011. A global optimization method for the design of space trajectories. *Computational Optimization and Applications* 48, 3, 635–652. DOI: http://dx.doi.org/10.1007/s10589-009-9261-6

[2] Farshid Amirabdollahian, Rui Loureiro, Elizabeth Gradwell, Christine Collin, William Harwin, and Garth Johnson. 2007. Multivariate analysis of the Fugl-Meyer outcome measures assessing the effectiveness of GENTLE/S

robot-mediated stroke therapy. *Journal of Neuroengineering and Rehabilitation* 4, 1, 4. DOI: http://dx.doi.org/10.1186/1743-0003-4-4

[3] Gustavo Andrade, Geber Ramalho, Hugo Santana, and Vincent Corruble. 2005. Challenge-sensitive action selection: An application to game balancing. In *Proceedings of the 2005 IEEE/WIC/ACM International Conference on Intelligent Agent Technology (IAT'05)*. IEEE, Los Alamitos, CA, 194–200. DOI: http://dx.doi.org/10.1109/IAT.2005.52

[4] Kleber De O. Andrade, Guilherme Fernandes, Glauco A. P. Caurin, Adriano A. G. Siqueira, Roseli A. F. Romero, and Rogerio De L. Pereira. 2015. Dynamic player modelling in serious games applied to rehabilitation robotics. In *Proceedings of the 2014 Joint Conference on Robitics: SWBR-LARS Robitics Symposium and Robocontrol (SBR LARS Robocontrol'14)*. 2011–2016. DOI: http://dx.doi.org/10.1109/SBR.LARS.Robocontrol.2014.41

[5] Kleber De O. Andrade, Thales B. Pasqual, Glauco A. P. Caurin, and Marcio K. Crocomo. 2016. Dynamic difficulty adjustment with evolutionary algorithm in games for rehabilitation robotics. In *Proceedings of the 2016 IEEE International Conference on Serious Games and Applications for Health (SeGAH'16)*. 1–8. DOI: http://dx.doi.org/10.1109/SeGAH.2016.7586277

[6] Kleber O. Andrade, Gisele G. Ito, Ricardo C. Joaquim, Bruno Jardim, Adriano A. G. Siqueira, Glauco A. P. Caurin, and Marcelo Becker. 2010. A robotic system for rehabilitation of distal radius fracture using games. In *Proceedings of the 2010 Brazilian Symposium on Games and Digital Entertainment (SBGAMES'10)*. IEEE, Los Alamitos, CA, 25–32. DOI: http://dx.doi.org/10.1109/SBGAMES.2010.26

[7] Maria-Virginia Aponte, Guillaume Levieux, and Stephane Natkin. 2009. Scaling the level of difficulty in single player video games. In *Proceedings of the 8th International Conference on Entertainment Computing (ICEC'09)*. 24–35. DOI: http://dx.doi.org/10.1007/978-3-642-04052-8

[8] A. Braga and C. Marinho. 2007. Jogos como sistemas: Análise dos elementos do jogo que divertem o jogador. In *Simpósio Brasileiro de Jogos e Entretenimento Digital*. São Leopoldo.

[9] Glauco A. P. Caurin, Adriano A. G. Siqueira, Kleber O. Andrade, Ricardo C. Joaquim, and Hermano I. Krebs. 2011. Adaptive strategy for multi-user robotic rehabilitation games. In *Proceedings of the Annual International Conference of the IEEE Engineering in Medicine and Biology Society (EMBS'11)*. DOI: http://dx.doi.org/10.1109/IEMBS.2011.6090328

[10] Ben Cowley, Darryl Charles, Michaela Black, and Ray Hickey. 2008. Toward an understanding of flow in video games. *Computers in Entertainment* 6, 2, Article No. 20. DOI: http://dx.doi.org/10.1145/1371216.1371223

[11] Mihaly Csíkszentmihályi. 2012. *Flow: The Psychology of Optimal Experience*. Harper Perennial Modern Classics.

[12] L. Davis. 1991. *Handbook of Genetic Algorithms*. Van Nostrand Reinhold. DOI: http://dx.doi.org/10.1.1.87.3586

[13] Kleber De O. Andrade, Guilherme Fernandes, Jose Martins, Viviane C. Roma, Ricardo C. Joaquim, and Glauco A. P. Caurin. 2013. Rehabilitation robotics and serious games: An initial architecture for simultaneous players. In *Proceedings of the ISSNIP Biosignals and Biorobotics Conference (BRC'13)*. 1–6. DOI: http://dx.doi.org/10.1109/BRC.2013.6487455

[14] A. E. Eiben and J. E. Smith. 2012. *Introduction to Evolutionary Computing Genetic Algorithms*. Springer. DOI: http://dx.doi.org/10.1007/978-3-662-05094-1

[15] L. J. Eschelman and J. D. Shaffer. 1993. Real-coded genetic algorithms and interval-schemata. *Foundations of Genetic Algorithms* 2, 187–202. DOI: http://dx.doi.org/citeulike-article-id:1045422

[16] Dimitris Giakoumis, Athanasios Vogiannou, Illka Kosunen, Kostantinos Moustakas, Dimitros Tzovaras, and George Hassapis. 2010. Identifying psychophysiological correlates of boredom and negative mood induced during HCI. In *Proceedings of the 1st International Workshop on Bio-Inspired Human-Machine Interfaces and Healthcare Applications*. 3–12. DOI: http://dx.doi.org/10.5220/0002812600030012

[17] Johan Hagelbäck and Stefan J. Johansson. 2009. Measuring player experience on runtime dynamic difficulty scaling in an RTS game. In *Proceedings of the 2009 IEEE Symposium on Computational Intelligence and Games (CIG'09)*. 46–52. DOI: http://dx.doi.org/10.1109/CIG.2009.5286494

[18] Ya'Nan Hao, Suoju He, Junping Wang, Xiao Liu, Jiajian Yang, and Wan Huang. 2010. Dynamic difficulty adjustment of game AI by MCTS for the game Pac-Man. In *Proceedings of the 2010 6th International Conference on Natural Computation (ICNC'10*, Vol. 8. 3918–3922. DOI: http://dx.doi.org/10.1109/ICNC.2010.5584761

[19] Neville Hogan and Dagmar Sternad. 2007. On rhythmic and discrete movements: Reflections, definitions and implications for motor control. *Experimental Brain Research* 181, 1, 13–30. DOI: http://dx.doi.org/10.1007/s00221-007-0899-y

[20] Robin Hunicke. 2005. The case for dynamic difficulty adjustment in games. In *Proceedings of the 2005 ACM SIGCHI International Conference on Advances in Computer Entertainment Technology*. 429–433. DOI: http://dx.doi.org/10.1145/1178477.1178573

[21] Ming-Shaung Ju, Chou-Ching K. Lin, Dong-Huang Lin, Ing-Shiou Hwang, and Shu-Min Chen. 2005. A rehabilitation robot with force-position hybrid fuzzy controller: Hybrid fuzzy control of rehabilitation robot. *IEEE Transactions on Neural Systems and Rehabilitation Engineering* 13, 3, 349–358. DOI: http://dx.doi.org/10.1109/TNSRE.2005.847354

[22] Mehran Kamkarhaghighi, Pejman Mirza-Babaei, Khalil El-Khatib, and Kathrin M. Gerling. 2017. Architecture guideline for game-based stroke rehabilitation. *World Journal of Science, Technology and Sustainable Development* 14, 2–3, 228–240. DOI: http://dx.doi.org/10.1108/WJSTSD-06-2016-0039

[23] Patricia Kan, Rajibul Huq, Jesse Hoey, Robby Goetschalckx, and Alex Mihailidis. 2011. The development of an adaptive upper-limb stroke rehabilitation robotic system. *Journal of Neuroengineering and Rehabilitation* 8, 1, 33. DOI: http://dx.doi.org/10.1186/1743-0003-8-33

[24] Kazuo Kawada, Hiroyuki Sogo, Toru Yamamoto, and Yasuhiro Mada. 2002. Evolutionary computation in designing a robust PD sway controller for a mobile crane. In *Proceedings of the 2002 International Conference on Control Applications*. 868–873. DOI: http://dx.doi.org/10.1002/wics.005

[25] Hermano Igo Krebs, Bruce T. Volpe, Dustin Williams, James Celestino, Steven K. Charles, Daniel Lynch, and Neville Hogan. 2007. Robot-aided neurorehabilitation: A robot for wrist rehabilitation. *IEEE Transactions on Neural Systems and Rehabilitation Engineering* 15, 1, 327–335. DOI: http://dx.doi.org/10.1109/TNSRE.2007.903899

[26] Konstantinos P. Michmizos and Hermano Igo Krebs. 2014. Pointing with the ankle: The speed-accuracy trade-off. *Experimental Brain Research* 232, 2, 647–657. DOI: http://dx.doi.org/10.1007/s00221-013-3773-0

[27] P. H. Mirvis and M. Csikzentmihaly. 1991. *Flow: The Psychology of Optimal Experience*. Vol. 16. Harper Perennial Modern Classics. DOI: http://dx.doi.org/10.5465/AMR.1991.4279513

[28] Jeffrey Peter Moffett. 2010. *Applying Causal Models to Dynamic Difficulty Adjustment in Video Games*. Worcester Polytechnic Institute.

[29] Andrew Noon. 2009. *Dynamic Difficulty Adjustment in Games: A Neuroevolutionary Approach*. Ph.D. Dissertation. http://www.sailorsetsail.co.uk/files/msc_diss.pdf.

[30] Reinhard Pekrun, Thomas Goetz, Lia M. Daniels, Robert H. Stupnisky, and Raymond P. Perry. 2010. Boredom in achievement settings: Exploring control-value antecedents and performance outcomes of a neglected emotion. *Journal of Educational Psychology* 102, 3, 531–549. DOI: http://dx.doi.org/10.1037/a0019243

[31] Brandon Robinson Rohrer. 2002. *Evolution of Movement Smoothness and Submovement Patterns in Persons With Stroke*. Ph.D. Dissertation. Massachusetts Institute of Technology, Cambridge, MA.

[32] Stefan Schaal. 2006. Dynamic movement primitives—a framework for motor control in humans and humanoid robotics. In *Adaptive Motion of Animals and Machines*. Springer, 261–280. DOI: http://dx.doi.org/10.1007/4-431-31381-8_23

[33] Pieter Spronck, Ida Sprinkhuizen-Kuyper, and Eric Postma. 2004. Difficulty scaling of game AI. In *Proceedings of the 5th International Conference on Intelligent Games and Simulation*. 33–37.

[34] Eric T. Wolbrecht, Vicky Chan, David J. Reinkensmeyer, and James E. Bobrow. 2008. Optimizing compliant, model-based robotic assistance to promote neurorehabilitation. *IEEE Transactions on Neural Systems and Rehabilitation Engineering* 16, 3, 286–297. DOI: http://dx.doi.org/10.1109/TNSRE.2008.918389

[35] Bin Wu, Dingding Chen, Suoju He, Qijin Sun, Zhengjun Li, and Minxi Zhao. 2011. Dynamic difficulty adjustment based on an improved algorithm of UCT for the Pac-Man game. In *Proceedings of the 2011 International Conference on Electronics, Communications, and Control (ICECC'11)*. 4255–4259. DOI: http://dx.doi.org/10.1109/ICECC.2011.6066649

[36] Shiou-Wen Yeh, Jia-Jiunn Lo, Jeng-Jia Huang, and Zhen-Yu Fan. 2007. Technologies for E-learning and digital entertainment. *Technologies for E-Learning and Digital Entertainment* 4469, 829–840. DOI: http://dx.doi.org/10.1007/978-3-540-73011-8

Received April 2017; revised October 2017; accepted October 2017